

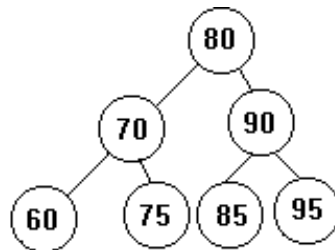
ÁRVORES BINÁRIAS DE PESQUISA

Árvores binárias de pesquisa são uma estrutura alternativa do tipo árvore binária, para guardar valores de tal forma que a recuperação dos mesmos pode ser efectuada de forma ordenada.

Assim, cada nó interno de uma árvore binária de pesquisa obedece aos seguintes critérios:

- Cada elemento tem uma chave, não existem chaves repetidas
- As chaves da subárvore esquerda (também árvore binária de pesquisa), de um qualquer nó considerado como raiz, são menores do que a chave dessa raiz
- As chaves da subárvore direita (também árvore binária de pesquisa), de um qualquer nó considerado como raiz, são maiores do que a chave dessa raiz

Abaixo representa-se uma árvore binária de pesquisa de inteiros. Nessa árvore cada nó conterá, além dos dois apontadores respectivamente para a subárvore esquerda e subárvore direita, um campo informação, designado por info e que no exemplo é ao mesmo tempo a chave.



Como se vê cada um dos nós internos segue os critérios anteriormente enunciados.

Verificamos também que se percorrermos uma árvore deste tipo, utilizando uma visita simétrica, obteremos os valores das chaves ordenados de forma crescente.

Assim a árvore acima representada dará origem à seguinte lista de valores:

60 70 75 80 85 90 95.

Seguidamente desenvolveremos os algoritmos de manipulação deste tipo de estrutura, nomeadamente, o pesquisar, o inserir novo nó e eliminar um determinado nó dado pela chave.

É evidente que os dois últimos algoritmos farão alterações à árvore, mas a nossa estrutura ter-se-á que manter binária de pesquisa

Algoritmo de PESQUISA

Pretende-se devolver o valor verdadeiro ou falso conforme se encontre ou não o nó com a chave que é levada como parâmetro.

Pesquisa(chave)

temp=raiz

enc=1 //flag que indica se o valor k foi ou não encontrado

Enquanto(*temp !=Nulo e enc==1*)

Se (*temp->info > chave*)

Então //pesquisa na subárvore esquerda

temp=temp->esq

Senão

Se (*temp->info < chave*)

Então //pesquisa na subárvore direita

temp=temp->dir

Senão // o conteúdo do nó é a nossa chave

Enc==0

Fse

Fse

Fenquanto

Se(*enc==1*)

Então *devolve falso*

Senão *devolve verdade*

Fse

Fpesquisa

Nesta estrutura as operações de pesquisa, juntar nó e eliminar nó serão de complexidade temporal $O(\log_2 n)$, e no caso menos favorável (árvore completamente desequilibrada, só com ramos direitos ou só com ramos esquerdos) de $O(n)$.

Como se verifica pelo algoritmo de pesquisa o número de comparações que no máximo se faz até se concluir que existe ou não a chave, é no máximo a altura da árvore. Se a árvore for (mais ou menos) equilibrada, os nós folha todos com a mesma profundidade, podemos relacionar a altura h da árvore, com o total de elementos n .

Assim,

$n = 2^{h+1} - 1$, ou seja,

$2^{h+1} = n + 1$

$h+1 = \log_2 (n+1)$

$h = \log_2 (n+1) - 1$,

assim podemos dizer que para todos os valores de $n \geq 1$, é possível encontrar uma constante C , tal que:

$\log_2 (n+1) - 1 \leq C * \log_2 n$, portanto o tempo de complexidade temporal do algoritmo de pesquisa é de ordem $\log_2 n$.

Para juntar um nó a uma árvore binária de pesquisa, fazemo-lo juntando esse nó a uma folha. Teremos que fazer uma pesquisa até atingirmos uma folha (caso a chave de pesquisa não exista). No caminho, desde a raiz até à folha, viramos para a subárvore esquerda ou direita conforme o valor da chave do nó a inserir é menor ou maior do que o valor da chave do nó que estamos a comparar na descida.

Abaixo encontra-se um algoritmo iterativo para juntar o nó com chave *ch*, passado como parâmetro

Juntar-no (*ch*)

temp=*raiz*

anterior=Nulo

enc=1 //flag que indica se o valor *k* foi ou não encontrado

Enquanto(*temp* !=Nulo e *enc*==1)

Se (*temp*->*info* > *ch*)

Então //pesquisa na subárvore esquerda

anterior=*temp*

temp=*temp*->*esq*

Senão

Se (*temp*->*info* < *ch*)

Então //pesquisa na subárvore direita

anterior=*temp*

temp=*temp*->*dir*

Senão // o conteúdo do nó é a nossa chave

Enc==0

Fse

Fse

Fenquanto

Se(*enc*==1)

Então // aloca-se espaço para o novoNo , coloca-se o conteúdo e os ponteiros *esq* e *dir* serão nulos, idto é feito pela função *cria_no*

novoNo=*cria_no*

Se (*anterior*=Nulo) //significa que a árvore era nula

Então *raiz*=*novoNo*

Senão //verifica-se se o novo no vai ficar do lado esquerdo ou direito da folha

Se (*anterior*->*info* > *ch*)

Então *anterior*->*esq*=*novoNo*

Senão *anterior*->*dir*=*novoNo*

Fse

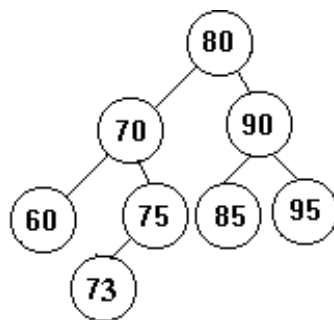
Fse

Fse

FJuntar_no

Este algoritmo poderia ser substituído por um algoritmo recursivo que deixamos como exercício.

Se à árvore que anteriormente representamos pretendêssemos juntar o nó com a chave 73, obteríamos a seguinte nova árvore:



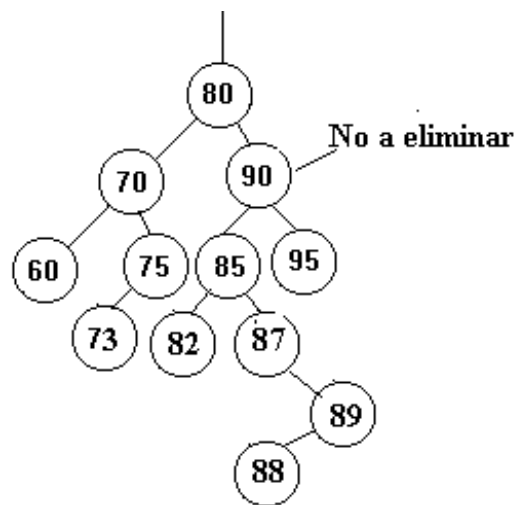
Para criarmos uma árvore binária de pesquisa a partir da árvore nula teremos que invocar o Juntar-no sucessivamente.

Para eliminarmos um nó de uma árvore binária de pesquisa, começaremos por procurar o nó então vários casos podem acontecer. O nó a eliminar :

1. É Folha (não tem as duas subárvores)
2. Falta-lhe uma das subárvores
3. Contem as duas subárvores.

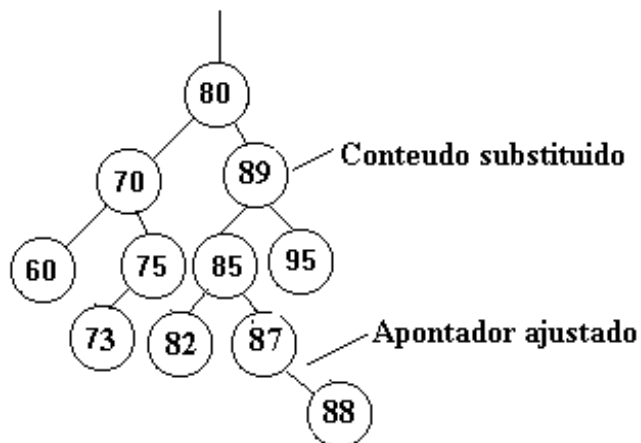
O caso 1 e 2 são facilmente resolvidos basta ajustar o apontador, do nó anterior (nó pai) que aponta para o nó que pretendemos eliminar. Fixamos o valor do apontador para o nó anterior quando efectuamos a pesquisa do nó a eliminar.

O caso 3, como facilmente se verifica ,torna-se mais complicado, se eliminássemos esse nó ficávamos com duas subárvores não sabendo onde as colocar. Seria o caso de eliminar o nó 90 na árvore abaixo desenhada



Assim, não se faz a eliminação desse nó, mas substitui-se o conteúdo do nó pelo conteúdo do nó imediatamente anterior em ordem simétrica e elimina-se este último, uma vez que este já não tem a subárvore direita. A árvore assim mantém-se binária de pesquisa.

Efectivamente o nó imediatamente anterior em ordem simétrica, é o nó que se encontra na subárvore esquerda do nó a eliminar (todas chaves são menores), o mais à direita possível (dos menores é o maior). No exemplo acima será o nó 89



Outra possibilidade seria a substituição do nó a eliminar pelo imediatamente seguinte em ordem simétrica seguida da eliminação deste último. Neste caso teríamos para substituir o conteúdo dos nós procurar na subárvore direita do nó a eliminar, o que se encontra o mais à esquerda possível.

No algoritmo que apresentamos a seguir para evitarmos vários *Ses* limitamo-nos a verificar se o nó a eliminar não tem a subárvore esquerda e nesse caso ajustamos o apontador do nó pai, caso contrário segue a regra do caso 3 (substituição do conteúdo).

Eliminar-no (ch)*temp=raiz**anterior=Nulo**enc=1 //flag que indica se o valor ch foi ou não encontrado***Enquanto**(*temp !=Nulo e enc==1*)**Se** (*temp->info > ch*)**Então** *//pesquisa na subárvore esquerda**anterior=temp**temp=temp->esq***Senão****Se** (*temp->info<chave*)**Então** *//pesquisa na subárvore direita**anterior=temp**temp=temp->dir***Senão** *// o conteúdo do nó é a nossa chave**Enc==0***Fse****Fse****Fenquanto***// termina a pesquisa***Se**(*enc==0*) *// temp aponta para o nó a eliminar e anterior para o pai deste***Então** *// verifica se não tem subárvore esquerda***Se** (*temp->esq==Nulo*)**Então** *// verifica se é raiz***Se** (*anterior==Nulo*)**Então** *temp=temp->dir**liberta raiz**raiz=temp***Senão** *// ajusta o apontador esquerdo ou direito do pai***Se** (*anterior->info < ch*)**Então** *anterior->dir=temp->dir***Senão** *anterior->esq=temp->dir***Fse***liberta temp***Fse****Senão** *// vamos encontrar o nó do filho esquerdo que está + à direita**filhoesq=temp->esq**nodir=filhoesq->dir***Se** (*nodir=Nulo*)

```
Então // troca conteúdos e ajusta apontador
    temp->info=filhoesq->info
    temp->esq=filhoesq->esq
    liberta filhoesq
Senão //descer à direita
    ant=filhoesq
    Enquanto(nodir->dir != Nulo)
        ant=nodir
        nodir=nodir->dir
    Fenquanto
        //troca conteúdos e ajusta apontador
        temp->info=nodir->info
        ant->dir=nodir->esq
        liberta nodir
    Fse
Senão
    Fse
    Feliminar_no
    Escreve(Não existe o nó a eliminar)
```

Tal como o nome indica este tipo de estrutura é muito utilizada em pesquisas, nomeadamente em tabelas de símbolos de compiladores e "assemblers" e em diferentes aplicações de bases de dados. Claro que pode ocasionalmente acontecer, devido à forma aleatória de inserir elementos e o retirar que conduza a tempos de resposta um pouco demorados, basta que a árvore se apresente desequilibrada. Este contratempo é eliminado com o uso de árvores equilibradas que serão estudadas posteriormente.